Zestaw 5

Zadanie 1. Oscylator harmoniczny, DSolve, PlotLegends

Rozwiąż równanie ruchu dla klocka o masie m przymocowanego do sprężyny o stałej sprężystości k. Ruch odbywa się bez tarcia powierzchniowego.

$$m \frac{\mathrm{d}^2 x}{\mathrm{d}\, t^2} = -k\, x$$

In[•]:=
```
OscillatorEquasion := m x''[t] == -k x[t]
```

Skorzystaj z polecenia DSolve:

1) Znajdź ogólną postać rozwiązania x[t] dla równania

In[•]:=
```
DSolve[OscillatorEquasion]
```

Out[•]=
$$\left\{\left\{x[t] \to c_1 \text{Cos}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right] + c_2 \text{Sin}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right]\right\}\right\}$$

Od ilu stałych zależy rozwiązanie tego równania?
Odp: 2

Podstaw za te stałe wybrane nie-zerowe wartości

In[•]:=
```
OscillatorEquasionWithArgs[m_, k_] := m x''[t] == -k x[t];
m0 = 2;
k0 = 3;
DSolve[OscillatorEquasionWithArgs[m0, k0]]
```

Out[•]=
$$\left\{\left\{x[t] \to c_1 \text{Cos}\left[\sqrt{\frac{3}{2}}\ t\right] + c_2 \text{Sin}\left[\sqrt{\frac{3}{2}}\ t\right]\right\}\right\}$$

2) Zadaj warunki początkowe jednoznacznie określające rozwiązanie x[t] dla równania

Ponownie rozwiąż z użyciem DSolve.
Użyj oznaczeń: prędkość początkowa = v0, położenie początkowe = x0.

```
In[•]:= solution = DSolve[{OscillatorEquasion, x[0] == x0, x'[0] == v0}, x, t]
```

$$\text{Out[•]= } \left\{\left\{x \to \text{Function}\left[\{t\}, \frac{\sqrt{k}\ x0\ \text{Cos}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right] + \sqrt{m}\ v0\ \text{Sin}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right]}{\sqrt{k}}\right]\right\}\right\}$$

3) Rozwiązanie z **2)** wykorzystaj do zdefiniowania funkcji `fx[t, m]` pokazującej położenie klocka po czasie t

Rozwiązanie z **2)** wykorzystaj do zdefiniowania funkcji `f[t, m]` pokazującej położenie klocka po czasie t dla ustalonych $x_0 = 1$, $v_0 = 0$, oraz $k = 1$, ale dowolnej masy m.
Przedstaw położenie klocków o masach odpowiednio $m = 0.5$, $m = 1$, $m = 2$ na wykresie dla t od 0 do 10.

```
In[•]:= fxBase[t1_][m_, k_, x0_, v0_] := Module[{solution},
          solution = DSolve[
            {m x''[t] == -k x[t], x[0] == x0, x'[0] == v0},
            x, t
           ];
          Last[First @ Flatten @ solution][t1]
         ]

        fx[t_, m_] = Module[{k, x0, v0},
          x0 = 1;
          v0 = 0;
          k = 1;
          fxBase[t][m, k, x0, v0]
         ]
```

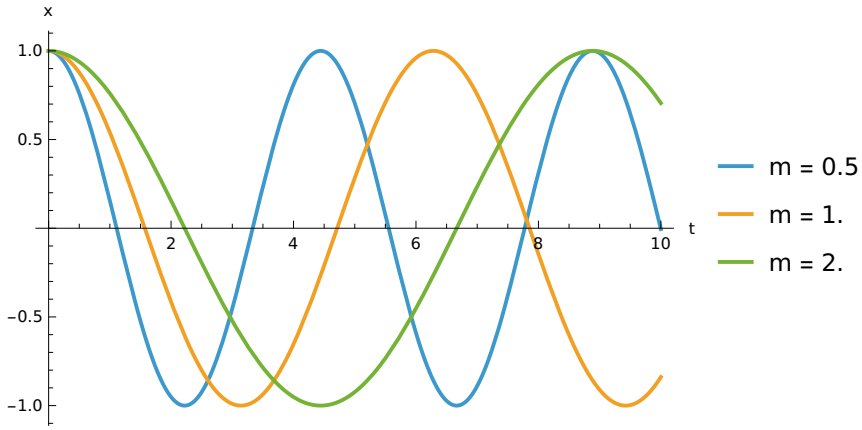
$$\text{Out[•]= } \text{Cos}\left[\frac{t}{\sqrt{m}}\right]$$

In[•]:=
```
masses = {1/2, 1, 2};

xFunctions = Map[
    m ↦ fx[t, m],
    masses
  ];

Plot[
 xFunctions,
 {t, 0, 10},
 AxesLabel → {"t", "x"},
 PlotLegends → Map[m ↦ "m = " <> ToString[N[m]], masses]
]
```

Out[•]=



4) Podobnie do 3) zdefiniuj funkcję `fv[t, m]` pokazującą prędkość klocka po czasie t

Podobnie do 3) zdefiniuj funkcję `fv[t, m]` pokazującą prędkość klocka po czasie t dla ustalonych $x_0 = 1$, $v_0 = 0$, oraz $k = 1$, ale dowolnej masy m.

In[•]:=
```
fvBase[t_][m_, k_, x0_, v0_] := D[fxBase[t][m, k, x0, v0], t];

fv[t_, m_] = Module[{k, x0, v0},
  x0 = 1;
  v0 = 0;
  k = 1;
  fvBase[t][m, k, x0, v0]
 ]
```

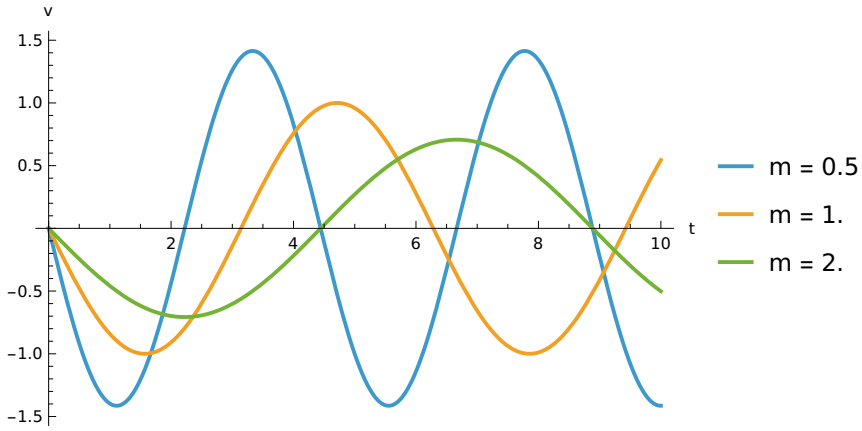
Out[•]=
$$-\frac{\text{Sin}\left[\frac{t}{\sqrt{m}}\right]}{\sqrt{m}}$$

Wykonaj analogiczny wykres jak w **3)** pokazujący prędkość klocka dla różnych mas.

In[•]:=
```
vFunctions = Map[
   m ↦ fv[t, m],
   masses
  ];

Plot[
 vFunctions,
 {t, 0, 10},
 AxesLabel → {"t", "v"},
 PlotLegends → Map[m ↦ "m = " <> ToString[N[m]], masses]
]
```

Out[•]=



5) Jaki jest okres wykonywanych drgań dla klocków o powyższych masach?

```
In[•]:= initialPositions = Flatten[Map[
          m ↦ Solve[fx[t, m] == 1, t, Reals],
          masses
        ]]
```

Out[•]= $\left\{t \to \sqrt{2}\ \pi c_1 \text{ if } c_1 \in \mathbb{Z},\ t \to 2\pi c_1 \text{ if } c_1 \in \mathbb{Z},\ t \to 2\sqrt{2}\ \pi c_1 \text{ if } c_1 \in \mathbb{Z}\right\}$

```
In[•]:= periods = Map[
          p ↦ First[Last[p]] / c1,
          initialPositions
        ]
```

Out[•]= $\left\{\sqrt{2}\ \pi,\ 2\pi,\ 2\sqrt{2}\ \pi\right\}$

6) Sprawdź, bez podstawiania wartości za x_0, v_0, k, m, czy w trakcie ruch klocka zachowana jest suma energii potencjalnej i kinetycznej

```
In[•]:= KineticEnergy[t_] = m fvBase[t][m, k, x0, v0]^2 / 2
```

Out[•]= $\frac{m\left(\sqrt{k}\ v0\ \text{Cos}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right] - \frac{k\ x0\ \text{Sin}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right]}{\sqrt{m}}\right)^2}{2k}$

```
In[•]:= PotentialEnergy[t_] = k fxBase[t][m, k, x0, v0]^2 / 2
```

Out[•]= $\frac{1}{2}\left(\sqrt{k}\ x0\ \text{Cos}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right] + \sqrt{m}\ v0\ \text{Sin}\left[\frac{\sqrt{k}\ t}{\sqrt{m}}\right]\right)^2$

```
In[•]:= FullSimplify[KineticEnergy[t] + PotentialEnergy[t]]
```

Out[•]= $\frac{1}{2}\left(m\ v0^2 + k\ x0^2\right)$

Zadanie 2. Oscylator harmoniczny tłumiony, DSolve, ParametricPlot

Rozwiąż równanie ruchu dla klocka o masie m przymocowanego do sprężyny o stałej sprężystości k przy założeniu, że opory ruchu są proporcjonalne do prędkości klocka ze współczynnikiem b.

$$m\frac{\mathrm{d}^2 x}{\mathrm{d}\,t^2} = -b\frac{\mathrm{d}\,x}{\mathrm{d}\,t} - k\,x$$

Skorzystaj z polecenia DSolve:

1) Zadaj warunki początkowe jednoznacznie określające rozwiązanie x[t] dla równania

Użyj oznaczeń prędkość początkowa = v0 oraz położenie początkowe = x0.

In[•]:=
```
DSolve[{
  m x''[t] == -b x'[t] - k x[t],
  x[0] == x0,
  x'[0] == y0
 }, x, t]
```

Out[•]=
$$\left\{\left\{x \to \text{Function}\left[\{t\}, \frac{1}{2\sqrt{b^2-4km}}\left(-b e^{\frac{1}{2}\left(-\frac{b}{m}-\frac{\sqrt{b^2-4km}}{m}\right)t} x0 + b e^{\frac{1}{2}\left(-\frac{b}{m}+\frac{\sqrt{b^2-4km}}{m}\right)t} x0 + e^{\frac{1}{2}\left(-\frac{b}{m}-\frac{\sqrt{b^2-4km}}{m}\right)t}\sqrt{b^2-4km}\, x0 + e^{\frac{1}{2}\left(-\frac{b}{m}+\frac{\sqrt{b^2-4km}}{m}\right)t}\sqrt{b^2-4km}\, x0 - 2e^{\frac{1}{2}\left(-\frac{b}{m}-\frac{\sqrt{b^2-4km}}{m}\right)t} m\, y0 + 2e^{\frac{1}{2}\left(-\frac{b}{m}+\frac{\sqrt{b^2-4km}}{m}\right)t} m\, y0\right)\right]\right\}\right\}$$

2) Rozwiązanie z **1)** wykorzystaj do zdefiniowania funkcji f[t, m, b] pokazującej położenie klocka po czasie t

Rozwiązanie z **1)** wykorzystaj do zdefiniowania funkcji f[t, m, b] pokazującej położenie klocka po czasie t dla ustalonych $x_0 = 1$, $v_0 = 0$, oraz $k = 1$, ale dowolnej masy m oraz współczynnika oporu b.

In[•]:=
```
fBase[t1_][m_, k_, b_, x0_, v0_] := Module[{solution},
   solution = DSolve[
     {
      m x''[t] == -b x'[t] - k x[t],
      x[0] == x0,
      x'[0] == v0
     },
     x, t];
   Last[First[Flatten[solution]]][t1]
  ];

f[t_, m_, b_] = Module[{k, x0, v0},
  x0 = 1;
  v0 = 0;
  k = 1;
  fBase[t][m, k, b, x0, v0]
 ]
```

Out[•]=
$$\frac{-b\, e^{\frac{1}{2}\left(-\frac{b}{m}-\frac{\sqrt{b^2-4m}}{m}\right)t} + b\, e^{\frac{1}{2}\left(-\frac{b}{m}+\frac{\sqrt{b^2-4m}}{m}\right)t} + e^{\frac{1}{2}\left(-\frac{b}{m}-\frac{\sqrt{b^2-4m}}{m}\right)t}\sqrt{b^2-4m} + e^{\frac{1}{2}\left(-\frac{b}{m}+\frac{\sqrt{b^2-4m}}{m}\right)t}\sqrt{b^2-4m}}{2\sqrt{b^2-4m}}$$

Przedstaw położenie klocka o masie $m = 1$ oraz wartościach:
$b = 0$ (bez tłumienia), $b = \frac{1}{10}$, $b = \frac{1}{2}$, $b = 4$, $b = 8$
na wykresie dla t od 0 do 20. Użyj `Plot`.

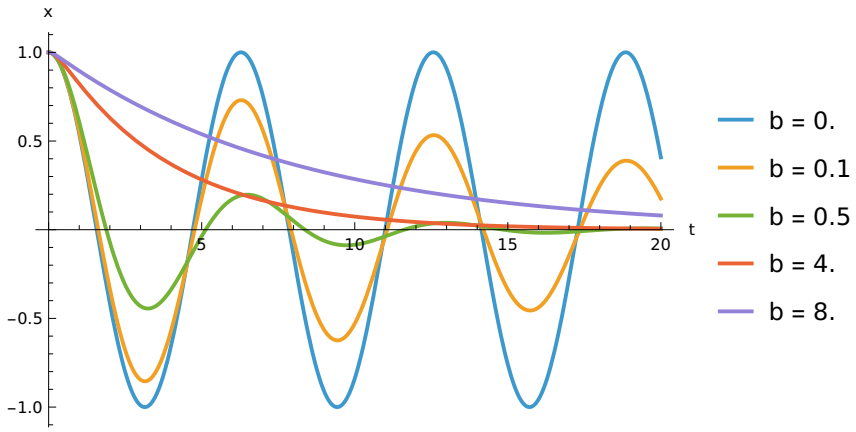
In[•]:=
```
bValues = {0, 1/10, 1/2, 4, 8};

PlotBFunctions[bValues_, tMin_, tMax_] := Module[{bFunctions},
   bFunctions = Map[
     b ↦ f[t, 1, b],
     bValues
    ];

   Plot[
    bFunctions,
    {t, tMin, tMax},
    AxesLabel → {"t", "x"},
    PlotLegends → Map[b ↦ "b = " <> ToString[N[b]], bValues]
   ]
  ];

PlotBFunctions[bValues, 0, 20]
```

Out[•]=



3) Wykonaj wykres fazowy dla $m = 1$ oraz wartości b jak w punkcie **2)**

W tym przypadku wykres fazowy to zależność prędkości od położenia klocka.

In[•]:=
```
fvbBase[t_][m_, k_, b_, x0_, v0_] := D[fBase[t][m, k, b, x0, v0], t];

fvb[t_, m_, b_] = Module[{k, x0, v0},
   k = 1;
   x0 = 1;
   v0 = 0;
   fvbBase[t][m, k, b, x0, v0]
  ];
```

Możesz użyć funkcji `ParametricPlot` by narysować krzywe $\left(x'(t), x(t)\right)$ dla odpowiednio dużego t.

In[•]:=
```
? ParametricPlot
```

Out[•]=

Symbol

ParametricPlot[{f_x, f_y}, {u, u_{min}, u_{max}}] generates a

parametric plot of a curve with x and y coordinates f_x and f_y as a function of u.

ParametricPlot[{{f_x, f_y}, {g_x, g_y}, ...}, {u, u_{min}, u_{max}}] plots several parametric curves.

ParametricPlot[{f_x, f_y}, {u, u_{min}, u_{max}}, {v, v_{min}, v_{max}}] plots a parametric region.

ParametricPlot[{{f_x, f_y}, {g_x, g_y}, ...}, {u, u_{min}, u_{max}}, {v, v_{min}, v_{max}}] plots several parametric regions.

ParametricPlot[{..., w[{f_x, f_y}], ...}, ...] plots

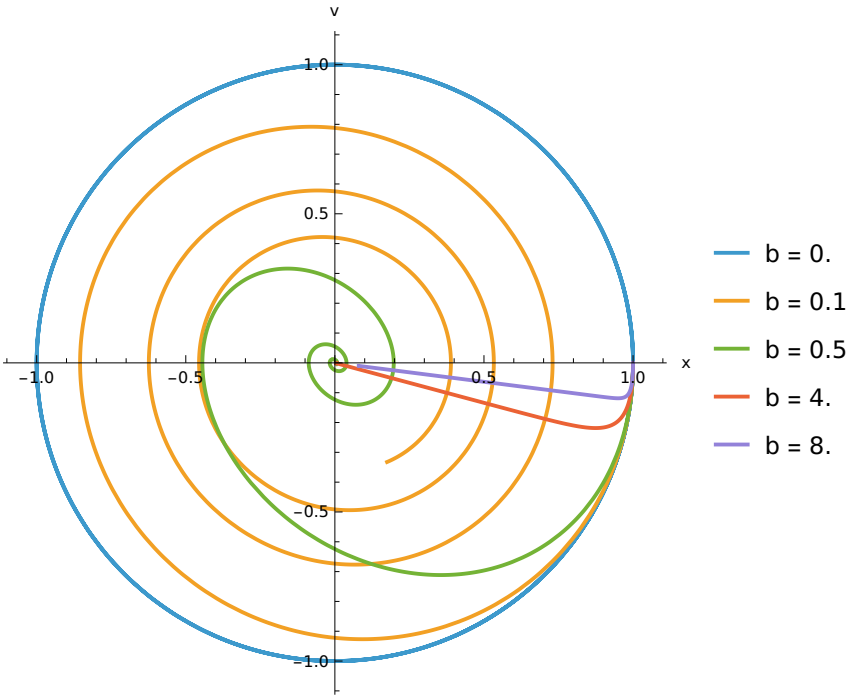
the curve {f_x, f_y} with features defined by the symbolic wrapper w.

ParametricPlot[..., {u, v} ∈ reg] takes parameters {u, v} to be in the geometric region reg.

In[•]:=
```
mass = 1;
paramFunctions = Map[
   b ↦ {f[t, mass, b], fvb[t, mass, b]},
   bValues
  ];

ParametricPlot[
 paramFunctions,
 {t, 0 , 20},
 AxesLabel → {"x", "v"},
 PlotLegends → Map[b ↦ "b = " <> ToString[N[b]], bValues]
]
```

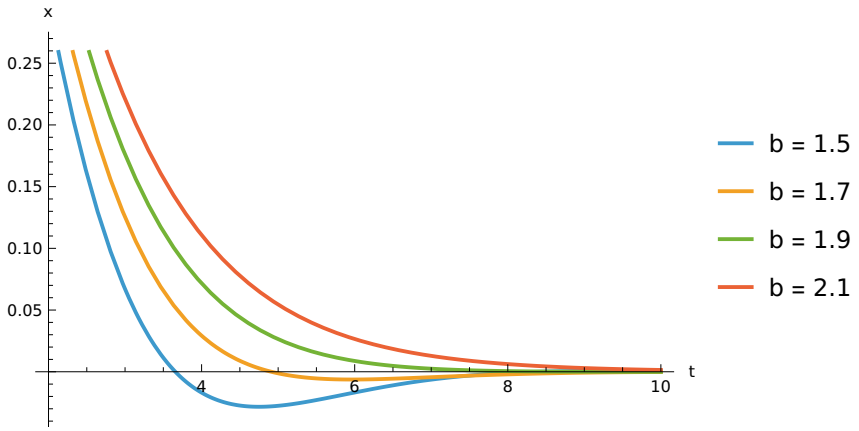
Out[•]=



4) Dla jakiej wartości współczynnika tłumienia b zmienia się jakościowy charakter ruchu?

In[•]:=
```
PlotBFunctions[{1.5, 1.7, 1.9, 2.1}, 2, 10]
```

Out[•]=



Odp:

- $b = 0$ - Brak tłumienia
- $b \in (0, 2)$ - Ruch harmoniczny tłumiony - Tłumienie podkrytyczne
- $b = 2\sqrt{m\,k} = 2$ - Tłumienie krytyczne
- $b > 2$ - Ruch opóźniony (“bloczek opadający w oleju”) - Tłumienie nadkrytyczne

5) Przedstaw na wykresie sumę energii potencjalnej i kinetycznej klocka

Przedstaw na wykresie sumę energii potencjalnej i kinetycznej klocka dla $m = 1$, $k = 1$, $x_0 = 1$, $v_0 = 0$, oraz zmieniającego się tłumienia $b = 0, \frac{1}{10}, \frac{1}{2}, 4, 8$.

In[•]:=
```
PotentialEnergy[t_, m_, k_, b_, x0_, v0_] = k fBase[t][m, k, b, x0, v0]^2 / 2;
KineticEnergy[t_, m_, k_, b_, x0_, v0_] = m fvbBase[t][m, k, b, x0, v0]^2 / 2;

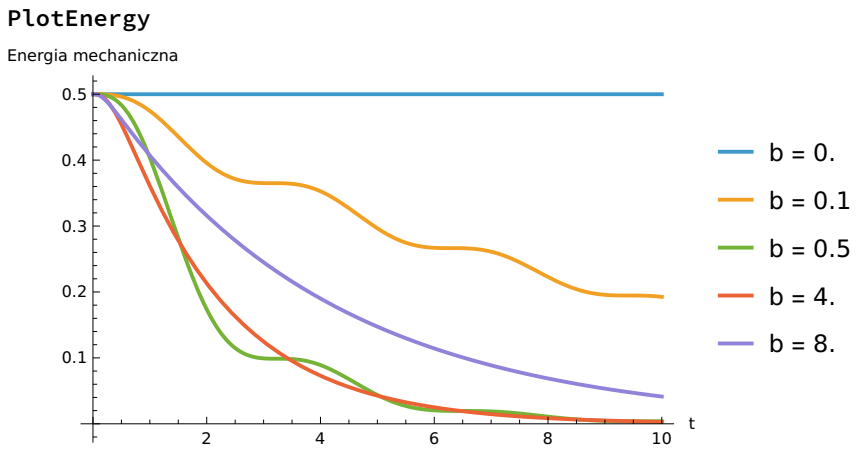
MechanicalEnergy[t_, m_, k_, b_, x0_, v0_] =
  PotentialEnergy[t, m, k, b, x0, v0] + KineticEnergy[t, m, k, b, x0, v0];
```

In[•]:=
```
PlotEnergy := Module[{m, k, x0, v0, bValues, totalEnergyFunctions},
  m = 1;
  k = 1;
  x0 = 1;
  v0 = 0;
  bValues = {0, 1/10, 1/2, 4, 8};

  mechanicalEnergyFunctions = Map[
    b ↦ MechanicalEnergy[t, m, k, b, x0, v0],
    bValues
   ];

  Plot[
   mechanicalEnergyFunctions,
   {t, 0, 10},
   AxesLabel → {"t", "Energia mechaniczna"},
   PlotLegends → Map[b ↦ "b = " <> ToString[N[b]], bValues]
  ]
 ]
```

Out[•]=



Czy energia jest zachowana?

Odp: Nie

Zadanie 3. Przybliżenie małych drgań, `Series`, Show, `PlotRange`, `PlotLegends`

W przypadku wahadła matematycznego złożonego z małej kulki o masie m zawieszonej na nitce o długości l, energię potencjalną kulki można wyrazić jako $E_{\text{pot}} = m\,g\,h$

gdzie:

- $h = l\left(1 - \cos\theta\right)$ - wysokość kulki nad położeniem równowagi,
- θ - kąt wychylenia wahadła,
- g – przyspieszenie grawitacyjne

Dla małych kątów θ, można przybliżyć $\frac{x}{l} = \sin\theta \approx \theta$

Podobnie dla małych kątów θ, można przybliżyć $\cos\theta \approx 1 - \frac{\theta^2}{2}$, skąd:

$$E_{\text{pot}} = m\,g\,h\,\frac{\theta^2}{2} = \frac{1}{2}\,m\,\frac{g}{l}\,x^2 = \frac{1}{2}\,\overline{k}\,x^2$$

gdzie $\overline{k} = m\,\frac{g}{l}$.

W zadaniu 1 okres drgań wynosił $T = 2\,\pi\,\sqrt{\frac{m}{k}}$.

Z kolei dla wahadła matematycznego po podstawieniu za k wartości $\overline{k}$ otrzymamy dobrze znany wynik $T = 2\,\pi\,\sqrt{\frac{l}{g}}$.

Takie podejście można zastosować do bardzo wielu układów, w których ruch odbywa się w potencjale kwadratowym albo potencjał można przybliżyć przez potencjał kwadratowy.

W tym zadaniu sprawdź w jakim stopniu funkcję $f(\theta) = 1 - \cos\theta$ można przybliżyć korzystając z rozwinięcia w szereg Taylora względem punktu $\theta_0 = 0$.

In[•]:=
```
Clear[f];
f[theta_] := 1 - Cos[theta];
theta0 = 0;
```

W tym celu:

1) Rozwiń $f(\theta)$ do 10-stopnia względem $\theta_0 = 0$ korzystając z funkcji `Series`

In[•]:=
```
? Series
```

Out[•]=

Symbol

Series[f, {x, x_0, n}] generates a power series expansion

for f about the point $x = x_0$ to order $(x - x_0)^n$, where n is an explicit integer.

Series[f, $x \to x_0$] generates the leading term of a power series expansion for f about the point $x = x_0$.

Series[f, {x, x_0, n_x}, {y, y_0, n_y}, ...] successively finds series expansions with respect to x, then y, etc.

In[•]:=
```
Series[f[theta], {theta, theta0, 10}]
```

Out[•]=
$$\frac{\text{theta}^2}{2} - \frac{\text{theta}^4}{24} + \frac{\text{theta}^6}{720} - \frac{\text{theta}^8}{40\,320} + \frac{\text{theta}^{10}}{3\,628\,800} + 0[\text{theta}]^{11}$$

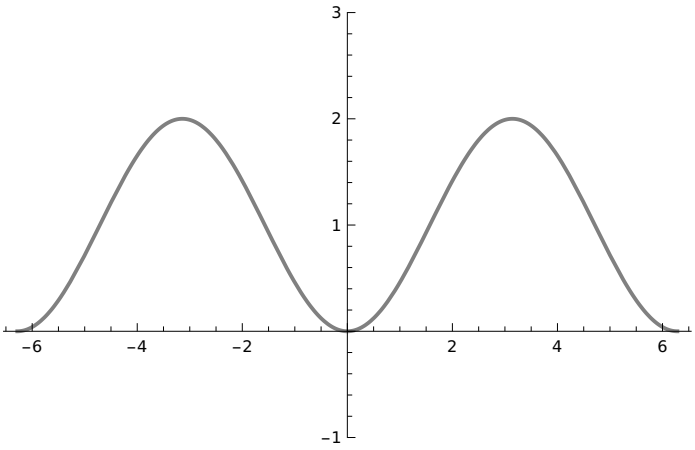
2) Przedstaw na wykresie funkcję $f(\theta)$

Przedstaw na wykresie funkcję $f(\theta)$ w zakresie od $-2\,\pi$ do $2\,\pi$, ogranicz z użyciem `PlotRange` zakres wartości y do $\{-1, 3\}$.
Na potrzeby kolejnego punktu, niech funkcja będzie częściowo przezroczysta, użyj `PlotStyle -> {Opacity[0.5, Black]}`.

In[•]:=
```
thetaRange = {theta, -2 Pi, 2 Pi};

plot = Plot[
  f[theta],
  thetaRange,
  PlotRange → {-1, 3},
  PlotStyle → {Opacity[0.5, Black]}
 ]
```

Out[•]=



3) Przedstaw przybliżenia funkcji $f(\theta)$ na powyższym wykresie

Wystarczą tylko parzyste stopnie, dlaczego?
Niech przybliżenia będą wyrysowane linią przerywaną. Wygodnie użyć funkcji Show.

In[•]:=
```
degrees = Range[2, 10, 2];

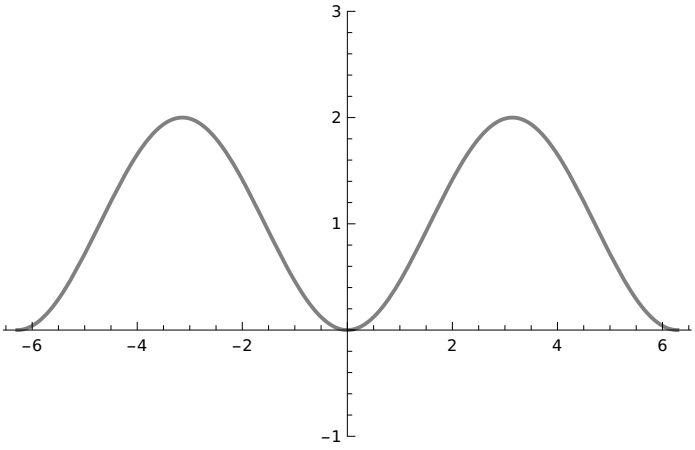
DegreeToFunction[degree_] := Module[{g},
  g[t_] = Normal[Series[f[t], {t, theta0, degree}]];
  g
 ];

aproxFunctions = Map[
  degree ↦ DegreeToFunction[degree],
  degrees
 ];

aproxPlot = Plot[
  Evaluate[Map[
    function ↦ function[theta],
    functions
   ]],
  thetaRange
 ];

Show[plot, aproxPlot]
```

Out[•]=

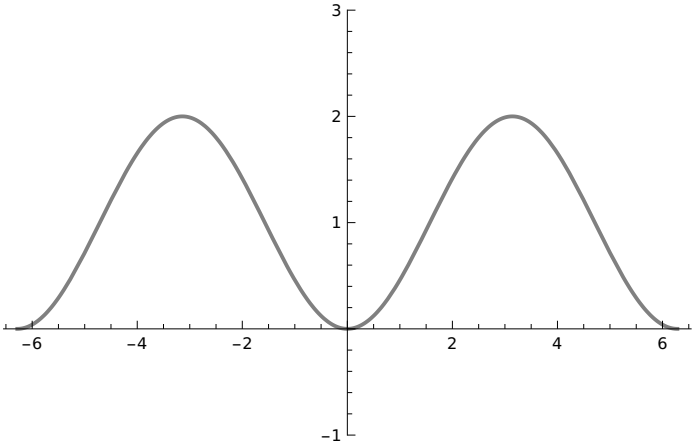


4) Dodaj legendę do wykresu korzystając z opcji PlotLegends -> "Expressions"

In[•]:=
```
aproxPlot = Plot[
   Evaluate[Map[
     function ↦ function[theta],
     functions
    ]],
   thetaRange,
   PlotLegends → "Expressions"
  ];

Show[plot, aproxPlot]
```

Out[•]=



5) Zbadaj jaka jest różnica $f(\theta)$ oraz przybliżeń kolejnych stopni w punkcie $\theta = \frac{\pi}{2}$

In[•]:=
```
theta1 = Pi / 2;

CalculateError[degree_] := Module[{AproxFunction, error},
  AproxFunction[t_] = Normal[Series[f[t], {t, theta0, degree}]];
  error = N @ Abs[
     (AproxFunction[theta1] - f[theta1]) / f[theta1]
    ];
  error
 ]

Map[
 degree ↦ degree → PercentForm @ CalculateError[degree],
 degrees
]
```

Out[•]=
```
{2 → 23.37%, 4 → 1.997%, 6 → 0.08945%, 8 → 0.002474%, 10 → 0.00004648%}
```

Dla jakiego stopnia przybliżenia różnica ta jest mniejsza od 1%?

In[•]:=
```
closeEnough = False;
n = 0;
result = {};

While[! closeEnough,
 n += 2;
 AproxFunction[t_] = Normal[Series[f[t], {t, theta0, n}]];
 error = CalculateError[n];

 closeEnough = error < 0.01;
]

Print["Stopień: " <> ToString[n]]
Print["Błąd: " <> ToString @ PercentForm[error]]
```

```
Stopień: 6
Błąd: 0.08945%
```